PORTFOLIO PRODUCT DOCUMENT

My Weather

A privacy-first, text-first weather utility

A complete product requirements document covering product strategy, scope, epics, features, user stories, acceptance criteria, architecture, security, validation, risks, and roadmap.

PRODUCT OWNER	Nikhil Sharma
ROLE	Product owner and builder
CURRENT BUILD	Version 1.5.0, build 7
STATUS	Working personal-use MVP
DOCUMENT DATE	October 13, 2026

Built with implementation assistance from ChatGPT models.

Executive summary

The product in one page: a narrow problem, a disciplined solution, and evidence of a working release.

PRODUCT PROPOSITION

My Weather gives one person a fast, quiet way to compare today's essential weather across up to five cities. One dark, text-first dashboard combines essential values with compact daily weather blocks, without drill-downs, launch fetching, location access, or account creation.

5	34	0	3
MAXIMUM SAVED CITIES	**AUTOMATED CORE TESTS**	**SENSITIVE PERMISSIONS**	**APPLE FORM FACTORS**
A deliberate focus constraint	36 tests in the iOS suite	No location or tracking	iPhone, iPad, and Mac

Problem

Mainstream weather products often prioritize maps, alerts, graphics, advertisements, location access, and dense forecast detail. The target user follows a small, stable set of cities and needs a faster task: open one screen, read today's essentials, and leave.

Solution

A native SwiftUI utility shows city name and local clock, current temperature, today's low/high, humidity, whole-day condition, and precipitation probability. Uniform labeled bars summarize the local day; a red dot marks the current period. Clocks update locally while cached weather changes only on an explicit refresh or city addition.

Current outcome

- Version 1.5.0 installed and launched on a physical iPhone; saved-data contents were verified unchanged after updating.
- Compatible app targets for iPhone and iPad on iOS/iPadOS 17+, and Mac on macOS 14+.
- Public MIT-licensed source with personal device builds through Xcode; no App Store or app account. Provider terms apply separately.
- Unsigned validation builds are used only to confirm compilation without accessing the owner's private signing certificate.

PORTFOLIO FRAMING

This case study demonstrates scope discipline, iterative simplification, privacy-by-design, resilient local caching, API-to-product translation, and transparent treatment of remaining product debt.

User, need, and strategy

The app is designed for a privacy-conscious individual who values speed and control more than feature breadth.

PRIMARY USER
A person who regularly monitors weather in a small set of home, family, work, or travel cities.

CORE NEED
Compare today's essential conditions quickly, without navigating, sharing location, or waiting for a launch refresh.

CONTEXT
Personal use across Apple devices, maintained directly through Xcode rather than an app marketplace.

Job to be done

CORE JOB
When I want to compare today's weather across important cities, I want every essential value on one screen so I can understand the day without navigating through multiple forecasts or granting location access.

Supporting jobs

User need	Product response
See last-known values immediately	Use the app before choosing whether to contact the network.
Add the correct city	Use region and country context to disambiguate shared names.
Know what a percentage means	Pair probability with Rain, Snow, Thunderstorm, or a general precipitation label.
Control data freshness	Refresh manually and see a clear last-updated timestamp.
Stay useful during outages	Preserve successful cached values until a later refresh succeeds.

Experience principles

Principle	How it shapes the product
Glanceability	All useful weather is on the main list. City rows do not navigate.
Intentional freshness	Routine fetching happens only after Refresh; adding a city is the explicit exception.
Resilience	A failed update never erases a successful snapshot.
Least privilege	No sensitive permissions, analytics, ads, login, or background activity.
Minimalism	Dark text-first interface, native controls, no decorative weather content.
Clear semantics	Technical codes and probabilities become concise human-readable labels.

Primary journey

1. Open	2. Review	3. Refresh	4. Manage
Cached cities and values render immediately.	Scan current, Low/High, humidity, precipitation, and local-day weather blocks.	Press Refresh only when newer data is desired.	Search, add, reorder, or delete; the list persists.

Scope, success, and release boundaries

The MVP succeeds by doing less - while making every included behavior reliable and explicit.

GOALS

- Show essential current and whole-day weather for up to five cities.
- Make worldwide search, add, delete, and reorder straightforward.
- Give the user explicit control over routine refreshes.
- Persist cities, order, snapshots, and update time across launches.
- Summarize the local day in up to six readable weather blocks.
- Minimize permissions, stored data, and network exposure.

NON-GOALS

- Separate city-detail, hourly-detail, or multi-day views.
- Detailed charts, maps, decorative weather art, or animation.
- Launch, foreground, timed, background, or pull-to-refresh fetching.
- Current-location detection, alerts, widgets, or notifications.
- Accounts, cloud sync, social sharing, ads, or analytics.
- Public App Store distribution in the personal MVP.

Success measures

Area	Portfolio success measure
Core task	A user can add a city and see its weather without leaving the main workflow.
Glanceability	All required values for five cities are available without drill-down.
User control	Zero weather requests occur merely because the app launched or foregrounded.
Persistence	Cities, order, values, and timestamp survive relaunch.
Resilience	A failed refresh never destroys a prior successful snapshot.
Privacy	Zero sensitive permissions and zero analytics or advertising SDKs.
Security	Accepted API responses originate from two exact HTTPS hosts.
Quality	Core mapping, validation, and persistence tests pass; supported targets build.
Accessibility	Core flows remain understandable with larger text and VoiceOver labels.

MEASUREMENT APPROACH

The personal MVP deliberately contains no product analytics. Success is evaluated through automated tests, device trials, build evidence, and direct task observation rather than telemetry.

Requirements map

Epic	Outcome	Status
E1 Saved city management	A focused, persistent, ordered set of up to five cities.	Implemented
E2 Weather acquisition	Current and daily values translated into clear language.	Implemented
E3 Single-screen dashboard	All useful weather visible without row navigation.	Implemented
E4 Persistence and resilience	Last-known-good values survive relaunch and failure.	Implemented
E5 Visual design and accessibility	Dark, compact, text-first native experience.	Implemented; manual QA remains
E6 Privacy and security	Least-permission app with restricted networking.	Implemented
E7 Quality and transparency	Validated build with honest source and distribution information.	Attribution follow-up
E8 Daily weather blocks	Compact local-day periods with readable times and gradients.	Implemented
E9 City clocks and Now	City-local clocks, aligned Now dots, and uniformly sized bars.	Implemented

EPIC 01

Saved city management

Create a small, durable set of relevant cities without duplicates or ambiguity.

WORLDWIDE CITY SEARCH — Implemented

MW-101 | As a user, I want to search by city name so I can add locations important to me.

Acceptance criteria

- Search appears at the top and is focused when eligible.
- No request before two trimmed characters; input is debounced about 300 ms.
- A newer query cancels the prior task; at most 10 populated places appear.
- Each result shows city plus region and country when available.
- Clear, Cancel, No Results, and Search Unavailable states are provided.

ADD AND PERSIST — Implemented

MW-102 | As a user, I want a selected city saved immediately so it is ready on my list.

Acceptance criteria

- Persist provider ID, name, region, country, coordinates, and timezone.
- Append after existing cities and close the Add City sheet.
- Trigger one refresh for all saved cities after a successful add.
- The saved city and order survive relaunch.

DUPLICATES AND LIMIT — Implemented

MW-103 | As a user, I want a focused list without repeated or excessive entries.

Acceptance criteria

- Use the provider city ID for duplicate detection.
- Existing results show Already Added and cannot be selected.
- Persist no more than five unique cities.
- Disable Add at five and explain that one city must be removed first.
- Defensively normalize corrupted or externally modified stored lists.

DELETE AND REORDER — Implemented

MW-104 | As a user, I want to manage priority and remove cities I no longer need.

Acceptance criteria

- Use standard list reordering and deletion behavior.
- Persist the resulting order without refetching weather.
- Deleting a city removes its cache and failure state.
- Deleting the final city clears the global update time.
- The visible list must not falsely appear saved after a persistence failure.

PRODUCT DECISION

The five-city ceiling is a design constraint, not a technical limitation. It protects glanceability and keeps requests bounded.

EPIC 02

Weather acquisition and interpretation

Fetch one local day of weather and translate provider data into concise, meaningful text.

MANUAL REFRESH — Implemented

MW-201 | As a user, I want a Refresh control so I decide when routine weather data changes.

Acceptance criteria

- Refresh exists only on the main screen.
- Disable it with zero cities and during an active refresh.
- Request every saved city concurrently within the connection limit.
- Collect all city responses before publishing one consolidated weather-state update.
- Do not fetch on launch, foreground, a timer, background activity, or pull gesture.

REFRESH AFTER ADD — Implemented

MW-202 | As a user, I want a new city populated immediately after I add it.

Acceptance criteria

- A successful add initiates a refresh for all cities.
- Existing cached values remain visible while requests run.
- Add and Refresh remain disabled during the operation.
- Each successful result replaces that city's older snapshot.

CURRENT AND WHOLE-DAY WEATHER — Implemented

MW-203 | As a user, I want today's essential values for each city's local day.

Acceptance criteria

- Show rounded current Celsius temperature and integer humidity.
- Show daily minimum and maximum; reject a minimum above the maximum.
- Use the saved city timezone for the forecast day.
- Translate daily WMO codes to Sunny, Mix, Cloudy, Fog, Rain, Snow, Thunderstorm, and related terms.
- Reject non-finite temperatures and percentages outside 0-100.

PRECIPITATION MEANING — Implemented

MW-204 | As a user, I want the percentage paired with what may happen.

Acceptance criteria

- Use the daily maximum precipitation probability.
- Prefer event labels such as Rain 60%, Snow 30%, or Thunderstorm 20%.
- Otherwise show Chance of Precipitation N%.
- If probability is absent, preserve the label and show a dash rather than inventing a value.
- Never expose raw numeric WMO codes.

DATA INTERPRETATION

The interface distinguishes current observations from daily forecast values, while keeping the wording compact enough for a five-city list.

EPIC 03

Single-screen weather dashboard

Make the main list the complete product experience and remove redundant navigation.

CITY ROW HIERARCHY — Implemented

MW-301 | As a user, I want a consistent sequence so I can scan several cities quickly.

Acceptance criteria

- Render city name first, then current temperature on its own line.
- Render Low and High next, followed by humidity and whole-day condition.
- Render precipitation type/chance last with reduced emphasis.
- Rows have no chevron, tap action, or detail destination.
- Group city values for accessibility and expose labeled weather periods separately.

STATES AND ACTIONS — Implemented

MW-302 | As a user, I want only essential actions and truthful empty states.

Acceptance criteria

- Show No Cities when the list is empty.
- Show Unavailable only when a city has no successful snapshot after failure.
- Toolbar contains About, Edit where applicable, Refresh, and Add.
- Disable Edit with fewer than two cities; enforce Add and Refresh eligibility.
- Do not add promotional copy, tutorials, or decorative empty-state art.

LAST UPDATED — Known limitation

MW-303 | As a user, I want to know when the visible weather was last refreshed.

Acceptance criteria

- Show Last Updated below the title and repeat it in About.
- Before success, display Last Updated: Not Yet.
- Format a successful time in the device locale/timezone, including its abbreviation when available.
- Launch and foreground events do not change the timestamp.
- Removing the final city resets the value.

KNOWN LIMITATION

The current global timestamp advances when at least one city succeeds. If another city keeps an older cached value, its age is not separately shown. The roadmap proposes per-city freshness or all-city timestamp semantics.

EPIC 04

Persistence and offline resilience

Keep the app useful before a refresh, after relaunch, and during partial or total network failure.

PERSIST CITIES — Implemented

MW-401 | As a user, I want my selected cities and order retained between sessions.

Acceptance criteria

- JSON encode city records into local UserDefaults.
- Persist only provider ID, display fields, coordinates, and timezone.
- Require no user account or cloud storage.
- Recover from corrupt city data without crashing.

PERSIST SNAPSHOTS — Implemented

MW-402 | As a user, I want last-successful weather displayed until newer data succeeds.

Acceptance criteria

- Store one successful snapshot per saved city and load it during initialization.
- Display cache immediately without initiating a network call.
- Overwrite only the city whose later request succeeds.
- Remove snapshots for deleted or no-longer-saved cities.
- Recover safely from corrupt cached state and migrate the legacy timestamp.

PARTIAL AND TOTAL FAILURE — Implemented

MW-403 | As a user, I want useful prior values protected when the service is unavailable.

Acceptance criteria

- A failed result never erases an existing snapshot.
- First-time failure with no cache results in Unavailable.
- Apply successful city results even when other cities fail.
- Ignore results for a city deleted while its request was in flight.
- Keep the saved-city list usable regardless of network status.

STORAGE BOUNDARY

UserDefaults is proportionate for at most five small records and snapshots. It is not encrypted, but the app stores no secrets or identity data. A versioned store would become relevant only if scope or schema complexity grows.

EPIC 05

Visual design and accessibility

Deliver a calm, native interface that remains readable without adding visual complexity.

DARK-ONLY VISUAL SYSTEM — Implemented

MW-501 | As a user, I want a consistent dark interface with no decorative complexity.

Acceptance criteria

- Always request dark appearance; provide no theme selector.
- Use native lists, sheets, spacing, and controls.
- Use compact labeled color blocks; exclude decorative art, maps, detailed charts, and animation.
- Provide a recognizable custom app icon.

COMPACT TYPOGRAPHY — Implemented

MW-502 | As a user, I want readable type that supports a dense but calm dashboard.

Acceptance criteria

- Prefer Verdana across app text.
- Use approximately 32 pt large title, 15 pt body/headline, 13 pt subheadline, and 11 pt footnote.
- Create fonts relative to Dynamic Type text styles.
- Do not convey critical meaning through color alone.

NATIVE CROSS-PLATFORM BEHAVIOR — Known limitation

MW-503 | As a user, I want the same core utility across my Apple devices.

Acceptance criteria

- Use one SwiftUI codebase for iPhone, iPad, and Mac.
- Target iOS/iPadOS 17+ and macOS 14+ in the app project.
- Use standard platform list, sheet, delete, and reorder behavior.
- Keep core flows usable at larger system text sizes.

CONSISTENT SHORT LABELS — Implemented

MW-504 | As a user, I want the same capitalization convention throughout the interface.

Acceptance criteria

- Use title-style capitalization in short interface and weather labels.
- Display Low Light, Sun / Cloud, Freezing Rain, Night Rain, and Last Updated consistently.
- Keep full sentences in sentence case and preserve proper city names.
- Compute display labels from unchanged category IDs so stored snapshots remain compatible.

VALIDATION GAP

Standard and largest Dynamic Type layouts were visually checked in the iPhone simulator. Relative text styles and VoiceOver labels are implemented, but automated UI/layout coverage and complete VoiceOver and iPad/Mac accessibility trials remain open.

EPIC 06

Privacy and security

Apply least privilege to permissions, networking, and local data while keeping the app understandable.

LEAST-PERMISSION OPERATION — Implemented

MW-601 | As a privacy-conscious user, I want weather without granting sensitive access.

Acceptance criteria

- Request no location, camera, microphone, photos, contacts, calendars, Bluetooth, or notification permission.
- Use no background execution, account, API key, ad identifier, analytics, or tracking SDK.
- On Mac, enable App Sandbox plus outbound network access only.
- Do not include sensitive UsageDescription keys or extra iOS entitlements.

RESTRICTED NETWORKING — Implemented

MW-602 | As a user, I want the app to contact only its declared provider.

Acceptance criteria

- Allow HTTPS to geocoding-api.open-meteo.com and api.open-meteo.com only.
- Reject alternate scheme, host, nonstandard port, username, or password.
- Accept only HTTP 200 application/json responses up to 1 MiB.
- Use 15 s request and 30 s resource timeouts.
- Use GET with an ephemeral session and no cookies, credentials, or URL cache.

MINIMAL LOCAL DATA — Implemented

MW-603 | As a user, I want only the data needed for this utility stored on my device.

Acceptance criteria

- Store saved city records, snapshots, and the refresh timestamp only.
- Store no precise device location, profile, usage history, or search analytics.
- Delete a city's snapshot when the city is removed.
- Clear the global timestamp when the final city is removed.

THREAT POSTURE

This is a low-data personal utility. The primary controls are attack-surface reduction, strict endpoint validation, bounded responses, defensive decoding, and no secret material in the app.

EPIC 07

Quality, transparency, and personal distribution

Make authorship, data sources, validation status, and the installation model explicit.

ABOUT AND AUTHORSHIP — Implemented

MW-701 | As a user, I want to know who made the app and how it was created.

Acceptance criteria

- Open About from an information icon.
- Display Nikhil Sharma and disclose help from ChatGPT models.
- Repeat the same Last Updated value as the main screen.
- Provide Done to dismiss; include no promotional copy or unrelated settings.

DATA-SOURCE TRANSPARENCY — Required follow-up

MW-702 | As a user, I want to understand where city and weather information originate.

Acceptance criteria

- Documentation identifies Open-Meteo forecast and geocoding services.
- City search is described as based on GeoNames data.
- The app explains that updated values require internet access.
- Keep source credits in the public README; add in-app attribution before public binary distribution.

BUILD AND INSTALL — Implemented

MW-703 | As the owner, I want a repeatable path from source to my Apple devices.

Acceptance criteria

- The Xcode project builds for supported iPhone-compatible and Mac destinations.
- Automatic signing uses the owner's selected Personal Team for a physical device.
- Running installs over the previous version when the bundle identifier is unchanged.
- Saved cities and cached values remain after an update.
- Personal Team provisioning is periodically renewed by rebuilding and reinstalling.

SIGNED VERSUS UNSIGNED BUILDS

An unsigned build proves the source compiles without using the owner's private Apple signing identity. It cannot be installed as the personal iPhone app. Xcode signs and installs the runnable device build when the owner's iPhone and Personal Team are selected.

EPIC 08

Compact daily weather blocks

Show the local day's progression beside each city without adding a detail screen or automatic refresh.

GLANCEABLE TIME PERIODS — Implemented

MW-801 | As a user, I want a small visual summary of when conditions change during the day.

Acceptance criteria

- Show at most six chronological blocks, combining adjacent similar periods.
- Each block shows a readable time range and weather label; no meaning relies on color alone.
- Use a 24-hour clock; 24 marks the midnight ending the day.
- Keep blocks beside city values when space permits; stack below at narrow widths or larger text sizes.
- Expose each period's time and condition to VoiceOver; add no row navigation.

WEATHER AND LIGHT CATEGORIES — Implemented

MW-802 | As a user, I want colors and labels that describe both light and significant weather.

Acceptance criteria

- Support Night, Dawn, Sunny, Sun / Cloud, Cloudy, Low Light, and Dusk.
- Include fog, drizzle, rain, freezing rain, snow, and storms, with night variants.
- Use gradients around light changes; describe transition times as approximate hourly summaries, not astronomical twilight.
- Weather events take precedence over light categories.
- Mark merged conditions as mixed and prioritize higher-severity events in short labels.

CITY-LOCAL DAY CORRECTNESS — Implemented

MW-803 | As a user, I want each city's times to describe that city's day, not my device's clock.

Acceptance criteria

- Use the saved IANA timezone and show the cached forecast date with a Local label.
- Cover the full local day, including 23/25-hour and half-hour daylight-saving transitions.
- Preserve minute offsets and disambiguate a repeated hour when needed.
- Keep the main Last Updated value in the device's local timezone.
- Hide incomplete or malformed optional block data while retaining valid basic weather.

CACHE AND RESOURCE DISCIPLINE — Implemented

MW-804 | As a user, I want this visual to remain useful offline without changing the app's privacy model.

Acceptance criteria

- Persist the optional timeline with each successful weather snapshot.
- Decode older snapshots with no timeline; fetch blocks on the next user-triggered refresh or city add.
- Use native SwiftUI views, with no downloaded images or additional dependencies.
- Add only required hourly/daylight fields and local-day padding to the existing city request.
- Add no permissions, launch refresh, background fetching, or secondary Refresh control.

DELIBERATE SCOPE CHANGE

The earlier MVP removed unused hourly detail. This iteration reintroduces only the hourly fields needed for a compact daily overview; separate detail screens remain out of scope.

EPIC 09

City-local clocks and Now markers

Add live local-time context without changing cached forecasts, network behavior, or the single-screen experience.

CITY-LOCAL CLOCK — Implemented

MW-901 | As a user, I want the current time and timezone next to each city name.

Acceptance criteria

- Show a zero-padded 24-hour HH:mm clock, followed by the relevant timezone abbreviation or GMT offset.
- Derive time from the saved IANA city timezone; honor daylight saving, half-hour, and quarter-hour offsets.
- Use IST for India; change seasonal abbreviations such as EDT/EST on the correct date.
- Keep the clock next to the city name when it fits; wrap below for long names or larger text.
- Show Time Unavailable for an invalid timezone instead of silently using the device timezone.

CURRENT-PERIOD INDICATOR — Implemented

MW-902 | As a user, I want a red dot to identify the city's current weather period.

Acceptance criteria

- Show one red dot only in the block whose absolute start/end interval contains now.
- Center the dot vertically and keep it at a fixed right inset, with a visible gap from the bar edge.
- Align dots across rows; their horizontal position does not represent elapsed progress.
- Use a light outline for contrast and include Now in the active bar's VoiceOver label.
- At an exact boundary select the next block; hide the dot outside the cached forecast day.

UNIFORM BAR DIMENSIONS — Implemented

MW-903 | As a user, I want every horizontal bar to have the same width and height.

Acceptance criteria

- Share one bar width and height across all currently saved cities, not per city or per label.
- Measure the longest time/condition labels and expand all bars equally when wrapping is needed.
- Reserve space for the dot even in inactive bars, so labels and geometry do not shift.
- Adapt the shared size to available width and Dynamic Type while preserving readable labels.
- Retain equal sizing when the bar stack moves below the weather values.

LOCAL-ONLY UPDATES — Implemented

MW-904 | As a user, I want live clocks without hidden weather refreshes or permissions.

Acceptance criteria

- Use a shared minute-level display schedule, not per-second polling or separate network requests.
- Advance clocks and Now selection without modifying saved weather or Last Updated.
- Keep cached weather visible until manual Refresh or a city addition triggers the existing refresh flow.
- Add no accounts, sensitive permissions, background modes, or dependencies.
- Preserve saved cities and cache when updating the same signed app installation.

TWO INDEPENDENT NOTIONS OF TIME

The clock shows the current local time in a city. Last Updated describes the last successful weather refresh in the device's timezone. A live clock does not mean that the saved weather was just refreshed.

Architecture and data contracts

A small layered design keeps SwiftUI, state management, networking, decoding, and local persistence independently testable.

SWIFTUI
ContentView - DayTimelineView - AddCityView - About

STATE
WeatherStore (@MainActor ObservableObject)

SERVICES
OpenMeteoWeatherService - ForecastBuilder - DayTimelineBuilder

BOUNDARIES
CityRepository / UserDefaults | Exact HTTPS Open-Meteo hosts

Data flow

Step	System behavior
1. Search	Trim and debounce query, call the geocoding endpoint, filter populated places, and deduplicate IDs.
2. Save	Persist the selected city's provider ID, display fields, coordinates, and IANA timezone.
3. Refresh	One combined current/hourly/daily request per city; fetch cities concurrently within connection limits.
4. Validate	Check transport constraints, array shapes, finite temperatures, numeric ranges, and low/high consistency.
5. Interpret	Map weather codes and light transitions, select the city's local day, and merge adjacent periods to at most six blocks.
6. Commit	After all city requests finish, publish one consolidated state update: replace successful snapshots, preserve failed cached values, and persist when any request succeeds.
7. Render	Show city values and city-local periods; the global Last Updated time stays device-local.

Core data contracts

Model	Persisted / consumed fields
SavedCity	id, name, region, country, latitude, longitude, timezone
CurrentWeather	temperature, humidity, plain-language condition
WeatherSnapshot	current weather, daily low/high, condition, precipitation, optional dayTimeline
DayTimeline / block	dateLabel, timezone, blocks; each block has start/end, time labels, and category IDs
WeatherDisplayState	snapshots, unavailable city IDs, and lastUpdated published together after refresh
StoredWeatherState	snapshots keyed by city ID, one optional global lastUpdated date

API contract

Endpoint	Request parameters	Consumed response fields
Geocoding /v1/search	name; count=10; language=en; format=json	results[].id, name, latitude, longitude, timezone, feature_code, country_code, country, admin1
Forecast /v1/forecast	latitude; longitude; timezone; past_days=1; forecast_days=2; timeformat=unixtime; temperature_unit=celsius	current time/temp/humidity/code; hourly time/code/is_day; daily time/min/max/code/precipitation max/sunrise/sunset; timezone and UTC offset

Security, privacy, and reliability

The product's security posture comes primarily from reduction: fewer permissions, fewer endpoints, fewer stored fields, and bounded responses.

Control area	Implemented control	User value
Permissions	No location or other protected-device permission declarations.	No sensitive system prompts or unnecessary access.
Transport	HTTPS only; exact allowlisted hosts; default port; no URL credentials.	Limits unexpected destinations and downgrade paths.
Response	Require 200 JSON, cap final payload at 1 MiB, validate decoded values.	Rejects oversized, malformed, or semantically invalid input.
Session	Ephemeral; no URL cache, cookies, credential storage, or response caching.	Reduces retained network state.
Timeouts	15-second request and 30-second resource limits.	Prevents unbounded waits.
Local data	Only chosen cities, coordinates, snapshots, and update time in UserDefaults.	No secrets, identity data, or behavior history.
App behavior	No analytics, ads, accounts, notifications, or background work.	Keeps the data flow narrow and predictable.
Mac sandbox	App Sandbox plus outbound network client only.	Constrains desktop capabilities.

Failure behavior

Scenario	Required outcome
App opens offline	Render cached cities and snapshots immediately; make no launch request.
One request fails	Keep that city's older snapshot while applying other successful results.
All requests fail	Keep old values and timestamp; a never-successful city becomes Unavailable.
City deleted in flight	Ignore its eventual result and remove its cache.
Corrupt local JSON	Fail safely to normalized empty city or weather state; do not crash.
Malformed provider values	Reject invalid arrays, non-finite temperature, out-of-range percentage, or inverted daily range.

Provider and license posture

- Open-Meteo provides the geocoding and forecast APIs used by the app; city search is based on GeoNames data.
- The selected API is keyless for qualifying non-commercial use, but provider terms, attribution, rate limits, and availability still apply.
- The free service has no guaranteed uptime. Cached values are therefore a product requirement, not merely an optimization.
- The public README credits providers. In-app attribution remains a follow-up before public binary distribution.

HONEST LIMITATION

If some cities fail while one succeeds, the current global timestamp can make older rows appear newer than they are. The app also lacks an offline/stale banner when cached values are retained.

Validation and measurement

Evidence is separated from aspiration: what is tested, what was observed, and what remains to validate.

Evidence	Current status
Automated tests	34 SwiftPM core tests and 36 iOS simulator tests passed for v1.5.0 on the sanitized public source.
Covered behaviors	Weather/persistence, mixed periods, severe-weather priority, optional hourly validation, old-cache compatibility, local DST days, clock abbreviations and midnight, Now boundaries/stale dates, cache immutability, and shared bar sizing.
Build validation	iOS Release and native Mac Release builds succeeded. The sanitized public project also passed an unsigned iOS Release build.
Unsigned Release bundle	2.719 MB (2.593 MiB), measured from v1.5.0 public-source .app file sizes. Signing, device thinning, and saved data can change the installed size.
Layout evidence	Standard and largest Dynamic Type iPhone layouts checked; native bar renders checked for uniform size and text clearance. Complete device accessibility review remains open.
Physical device	Version 1.5.0 (build 7) installed and launched; before/after saved-data checksums matched.
Not yet automated	UI layout, accessibility, WeatherStore partial failure, request construction/host security, live API integration, and Mac-specific behavior.

VERIFICATION BOUNDARY

Automated test counts, successful compilation, visual review, and physical installation are distinct evidence. Passing unit tests does not imply full accessibility coverage, production service guarantees, or App Store readiness.

Iteration and decision record

The product evolved through explicit user feedback, keeping each scope change narrow and testable.

Iteration narrative

Stage	Observed need	Product change
Prototype	See essential values without graphics.	Built a native dark city list with current and daily data.
Semantics	A bare percentage was ambiguous.	Changed precipitation into event-specific language.
Usability	Search at the bottom felt unintuitive.	Moved search to the top and added focused states.
Freshness	The user wanted explicit control.	Removed launch/foreground fetch and kept manual Refresh.
Resilience	Weather should remain visible between refreshes.	Persisted last-known-good snapshots and timestamp.
Simplification	Detail content duplicated the dashboard.	Removed city-detail navigation and secondary Refresh.
Hardening	Personal data exposure should be minimal.	Restricted hosts, session state, payload size, permissions, and entitlements.
Efficiency	Avoid unused data and repeated rendering work.	v1.3 removed unused hourly detail, batched refresh publication, and reused the Last Updated formatter.
Daily visual	See when light and weather change without opening details.	v1.4 added a bounded local-day block stack; only required hourly fields returned, in the existing city request.
Consistency	Short labels used inconsistent capitalization.	v1.4.1 standardized title-style labels, including Low Light and Sun / Cloud, with regression tests.
Local context	Locate each city in its day at a glance.	v1.5.0 adds city clocks and fixed right-aligned Now dots; all bars share width and height.

Decision record

Decision	Rationale and tradeoff
Manual refresh	Maximizes user control and avoids hidden network activity; data can become stale until refreshed.
No location	City search fully meets the use case with less privacy exposure; there is no automatic nearby weather.
Maximum five	Protects focus and request bounds; not designed for large watchlists.
Cache last success	Improves offline usefulness; requires honest freshness communication.
One-screen experience	Removes duplicate navigation; intentionally excludes deeper forecast exploration.
Daily blocks	Adds timing context without another screen; at most six merged periods, so short events may be summarized rather than shown individually.
Live clocks, saved weather	Minute-level local clocks and active-period markers do not fetch weather or change Last Updated. Old-day forecasts keep their values but show no Now dot.
Dark-only Verdana	Matches the personal design brief; reduces theme and font customization.
UserDefaults	Proportionate for a tiny local data set; not a general-purpose or encrypted database.
Open-Meteo	Keyless worldwide data for qualifying use; introduces provider availability and attribution obligations.

Risks, roadmap, and release procedure

The next work should improve correctness and evidence before expanding the feature set.

Risk	Impact	Mitigation / next action
Partial refresh timestamp	Older rows may appear as fresh as a newly updated row.	Add per-city freshness or advance global time only after full success.
Missing in-app attribution	Compliance follow-up before public binary distribution.	Public README already credits sources; add visible Open-Meteo/GeoNames credits in About.
Provider outage or terms change	Fresh data unavailable or use assumptions change.	Retain cache; review provider terms before broader/commercial use.
Personal Team expiry	Installed app may need a fresh signature.	Rebuild and reinstall through Xcode, or use a paid developer membership.
Accessibility gaps	Large text or VoiceOver issues may escape core tests.	Run device QA and add UI/accessibility coverage.
Platform declaration drift	Package and app target minimums can diverge.	Align macOS deployment targets and installation documentation.

Prioritized roadmap

P0 - CORRECTNESS AND COMPLIANCE

- Add visible provider attribution.
- Fix freshness semantics for partial success.
- Show a concise stale/offline state when cache is retained.

P1 - QUALITY EVIDENCE

- Add accessibility and partial-failure tests.

P2 - EVIDENCE-LED ONLY

- Consider optional features only after direct need is observed.
- Preserve the no-account, no-location, one-screen product boundary.
- Do not add charts or detail views without a new validated job.

Update the app on the iPhone

Step	Action
1	Open MyWeather.xcodeproj. Xcode normally detects externally edited source automatically.
2	If Xcode says files changed outside the app, choose Use Version on Disk.
3	Confirm MyWeather target, automatic signing, and your own Apple Account team. Keep signing changes local.
4	Connect and unlock the iPhone with Developer Mode enabled; select it as the run destination.
5	Press Run (Command-R). Xcode builds, signs, installs, and launches the update.
6	Keep the existing bundle identifier and signing team to update the installed app and retain saved cities/cache.
7	With a free Personal Team, repeat when provisioning expires.

Sources and appendix

Primary product and platform references used to frame the portfolio requirements.

My Weather - public source and MIT license
https://github.com/nikhilrsharma/my-weather

Open-Meteo Weather Forecast API
https://open-meteo.com/en/docs

Open-Meteo Geocoding API
https://open-meteo.com/en/docs/geocoding-api

Open-Meteo pricing and usage limits
https://open-meteo.com/en/pricing

Open-Meteo terms and attribution
https://open-meteo.com/en/terms

Open-Meteo model and feature overview
https://open-meteo.com/en/features

Apple - Run an app on simulated or physical devices
https://developer.apple.com/documentation/Xcode/running-your-app-on-simulated-or-physical-devices

Release-candidate checklist

Area	Checklist
Functional	Search ambiguous cities; enforce duplicate and five-city rules; add, delete, reorder; verify no row navigation.
Freshness	Confirm no launch/foreground request; verify manual and add-triggered refresh; test partial failure semantics.
Persistence	Relaunch with cities and cache; confirm overwrite; corrupt stored payload; delete final city.
Layout	Physical iPhone; iPhone simulator; iPad portrait/landscape; native Mac or Catalyst; long names; dark appearance.
Weather blocks	Check city clocks, DST, midnight, stale-day marker suppression, equal bar sizes, right-inset dot alignment, mixed labels, and cache retention.
Accessibility	Large Dynamic Type, VoiceOver reading order, toolbar labels, and contrast.
Security	Inspect entitlements and permission prompts; verify exact hosts, HTTPS, response limits, and ephemeral session.
Compliance	Add and verify Open-Meteo/GeoNames attribution before any public build.
Build	Run all tests; build supported targets; install signed physical-device build; verify retained data.

PORTFOLIO SUMMARY

My Weather is intentionally small, but the product work is complete in the dimensions that matter: a clear user job, disciplined scope, measurable acceptance criteria, resilient data behavior, privacy-first implementation, iterative simplification, and an evidence-based roadmap.

Document owner: Nikhil Sharma

Product: My Weather v1.5.0 (build 7) | PRD revision 1.3 | October 13, 2026

Implementation assistance disclosed: ChatGPT models.